Research Statement

Rokib Al Dhin Raadh · raadh.me · raadh@oxiedo.com · a16z Frontier Research Intern, Summer 2027

The question

Can we know, while a model is training, that training is still sound, and prove it rather than guess it?

Why it is open

Almost every convergence guarantee in deep learning assumes each update is a gradient step on a fixed loss. Real training breaks that assumption constantly. Gradient clipping, optimizer momentum, learning-rate schedules, data mixing, RL fine-tuning against a shifting reward, and self-repair all produce updates that are not gradients of anything. We train frontier models on mechanisms our theory does not cover, and we find failures afterwards: a loss spike that wastes a week of compute, a dead or saturated component, a reward hack that only shows up in evaluation.

Why it is hard

The loss landscape is non-convex, so global guarantees are mostly out of reach. Non-gradient updates remove the usual tool, the loss itself as a Lyapunov function. And any check has to run inside a training job without slowing it down or drowning engineers in false alarms.

Why it matters

Training is the most expensive and least observable step in building AI. A certificate that says "this run is stable as long as these quantities stay bounded", or "these units can never recover", turns monitoring from dashboards into decisions. It also matters for safety and regulation: an auditable record of what changed during training is the basis for trusting a model.

What I have done

I have completed one full turn of this loop, alone.

- **Theory.** In my DeepMath 2026 paper (sole author, double-blind) I treated self-repair as a bounded disturbance and proved local practical input-to-state stability, a control-theory guarantee that survives non-gradient updates. The same analysis gave a hard negative result: bounded per-channel repairs cannot revive a dead BatchNorm–ReLU channel. OpenReview
- **Tool.** I turned that result into ormas-monitor (Apache-2.0, PyPI). On pretrained models it found 292 channels in VGG-19-BN that output exactly zero on all 10,000 ImageNetV2 photos; removing them cut compute by 18.6% and changed no predictions, while removing 292 random live channels changed 499 of 500. Write-up
- **Evidence.** 383 controlled, seeded experiments on one RTX 3090, all reproducible from code. I presented the work as an invited talk at Cohere Labs and published corrections to my own results there.

What I would do at a16z Frontier

My results are on CNNs; the open question lives in transformers and RL post-training. In the internship I would (1) derive stability conditions for common non-gradient updates such as clipping and RL fine-tuning, (2) turn each condition into a cheap runtime check, and (3) test those checks on real training runs, including failed ones, to measure how early they warn and how often they are wrong. The output would be a paper, an open-source release and a public write-up. a16z Frontier offers what I lack: researchers who bridge theory and practice, compute, and portfolio companies whose training runs are exactly the data this research needs.